

RockHard: An AI-Powered Building Facade Climbing Route Analyser

Course: 20.318 Creative Machine Learning

Term: Term 6

Team Members:

- 1008271 Nigel Quek
- 1008031 Bethanie Khoo
- 1008203 Lim Han Xue
- 1008458 Muhammad Hazim

Abstract

RockHard is an end-to-end machine learning system that transforms ordinary building facades into analysed climbing environments. Given a photograph of a building, the system automatically detects existing climbable architectural features — ledges, windowsills, protrusions, ornaments, and pipes — constructs a weighted graph over those features, finds an optimal climbing route using A* pathfinding informed by learned biomechanics, grades the route on the standard V-scale, and generates a photorealistic visualisation of the facade with integrated climbing features. The system is deployed as an interactive web application allowing users to upload facade images, manually mark or auto-detect holds, generate and adjust routes, target specific difficulty grades, and export redesign renders. This report presents the full technical architecture, machine learning components, algorithmic design, and implementation details of the system.

1. Introduction

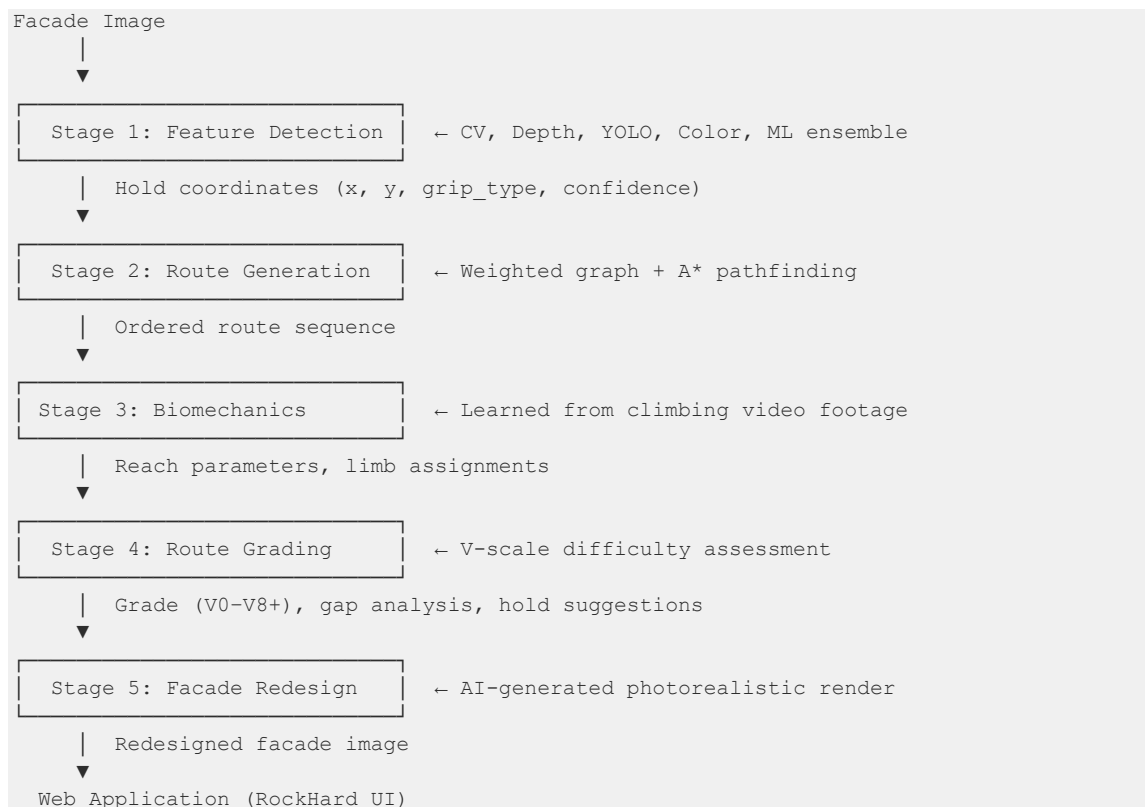
Urban climbing — the practice of ascending building facades — exists at the intersection of architecture, sport, and accessible design. Historically, buildings have not been designed with climbing in mind. Yet many existing facades contain inherent architectural features (projecting sills, decorative cornices, exposed pipework, textured surfaces) that a trained eye can read as a navigable climbing surface. Formalising this intuition as a computable problem — detecting climbable features, evaluating their reachability, grading the resulting route, and suggesting modifications — is the central goal of RockHard.

The project has direct practical application. Builders, architects, and urban developers can use the tool to assess whether an existing facade can be adapted for recreational climbing, understand what supplementary holds are needed to hit a target difficulty grade, and visualise the architectural changes as photorealistic renders before any physical modification occurs. This makes the tool useful both as a design instrument and as a communication aid for stakeholders.

The system brings together computer vision, depth estimation, graph-based pathfinding, machine learning classification, biomechanics analysis from video, route grading, and generative image synthesis — all unified into a single interactive pipeline.

2. System Overview

The RockHard pipeline consists of five sequential stages, each producing structured outputs that feed into the next:



All stages are accessible through an interactive Flask web application running locally, allowing non-technical users to engage with the full pipeline through a browser interface.

3. Stage 1 — Hold Detection

Detecting climbable features from a single facade photograph is a challenging multi-class detection problem. Climbable features span a wide range of appearances — a projecting sill looks fundamentally different from a decorative bracket or an exposed pipe — and many share visual similarity with non-climbable architectural details. Rather than relying on a single detection method, the system implements five independent detectors and fuses their outputs through weighted ensemble voting.

3.1 Computer Vision Detection

The primary detector operates entirely on classical computer vision techniques without any learned model. Processing proceeds as follows:

1. **Preprocessing:** The input image is converted to grayscale and enhanced using Contrast-Limited Adaptive Histogram Equalisation (CLAHE) to improve local contrast, making subtle protrusions more visible.
2. **Edge Detection:** Adaptive thresholding followed by Canny edge detection extracts structural boundaries from the image. Morphological opening and closing operations clean the binary edge map, removing noise while preserving structural contours.
3. **Contour Analysis:** Connected contours are extracted and filtered by area (minimum 80px²), aspect ratio, and solidity. Each surviving contour is treated as a candidate feature.
4. **Grip Classification:** Each candidate is classified into one of eight grip types — *jug, hold, sill, ledge, protrusion, crimp, ornament, pipe* — using a rule-based classifier that considers the contour's aspect ratio, solidity, area, and position within the image. For example, wide flat contours with high solidity near horizontal edges are classified as sills or ledges; compact circular contours are classified as jugs or ornaments; elongated near-vertical contours suggest pipes.
5. **Output:** A list of detected features, each with pixel coordinates (x, y), bounding dimensions, area, aspect ratio, solidity, and classified grip type.

3.2 Depth-Based Detection

Many architectural protrusions that are not visually distinct in a 2D photograph become clearly apparent when depth information is available. The depth-based detector extracts a depth map from the facade image and analyses it for geometric protrusions.

Depth Estimation: The system uses *Depth Anything V2*, a monocular depth estimation model, to generate a dense per-pixel depth map from a single RGB image. Where GPU resources are insufficient, an OpenCV Laplacian-based sharpness proxy is used as a fallback.

3D Protrusion Analysis: Using depth maps alongside the MoGe pointmap model, the system constructs a 3D point cloud of the facade surface. A local neighbourhood comparison identifies pixels whose depth value significantly exceeds the surrounding wall plane — these represent physical protrusions. The protrusion magnitude is normalised and thresholded to produce candidate hold locations.

Surface Normal Estimation: Surface normals are computed from the pointmap and used to filter out detections on non-climbable surfaces (e.g. near-horizontal overhangs, steep inward-facing surfaces) and confirm the orientation of candidate holds.

Generated depth maps and protrusion visualisations are cached in `outputs/depth_maps/` to avoid recomputation.

3.3 YOLO Object Detection

A YOLOv8 model was trained on a custom dataset of annotated facade images to directly detect climbable features as bounding boxes. The training dataset, located at `data/yolo_dataset/`, consists of approximately 15–20 manually annotated facade images split into train and validation sets. Annotations mark instances of ledges, sills, protrusions, ornaments, and pipes with bounding box labels in standard YOLO format. The trained model weights are stored at `models/yolo_hold_detector.pt`. Detection confidence scores are passed downstream as part of the ensemble.

3.4 Colour-Based Segmentation

For artificial climbing walls where holds are commercially manufactured and painted in standardised colours, a colour-based segmentation approach is applied. The image is converted to HSV colour space and thresholded for eight common hold colours: orange, red, pink, blue, teal, green, yellow, and purple. Connected regions above a minimum area threshold are classified as holds. This detector is primarily useful for validation against ground-truth artificial walls and less applicable to building facades.

3.5 Ensemble Fusion

Detections from all four methods above are merged through a weighted voting scheme:

Detector	Weight
Computer Vision	0.20
Depth / Protrusion	0.30
ML Classifier	0.20
YOLO	0.30

Candidate detections within a spatial proximity threshold are grouped, and their weighted confidence scores are aggregated. Candidates below a minimum combined confidence of 0.30 are discarded.

Terrain Filtering: A terrain segmentation module classifies each pixel into one of three regions — sky (top), facade (middle), ground (bottom) — using HSV and LAB colour analysis combined with vertical position priors. Detections in the sky or ground regions are removed as false positives, as these areas cannot realistically host climbable features.

3.6 Output Format

Final detections are stored as CSV files with the following columns:

```
x, y, width, height, area, aspect_ratio, solidity, grip_type, confidence
```

4. Stage 2 — Route Generation

Given a set of detected features on the facade, the route generation stage finds the optimal climbing path from the bottom to the top of the facade. This is formulated as a shortest-path problem on a weighted graph.

4.1 Graph Construction

Each detected feature becomes a node in a directed graph. An edge is added between two features if and only if the distance between them is within a configurable maximum reach parameter (default: 200 pixels). Edge weights are computed by the function `compute_edge_weight()`, which combines several factors:

Distance penalty: The base weight is proportional to the square of the Euclidean distance between holds, strongly penalising long reaches. This reflects the exponentially greater difficulty of extended moves in climbing.

Grip difficulty multiplier: Each grip type carries an inherent difficulty factor reflecting how hard it is to hold:

Grip Type	Difficulty Factor
Added Hold	0.8
Sill	1.0
Ledge	2.0
Protrusion	2.5
Crimp	3.0
Pipe	3.5
Ornament	4.0

Directional modifiers:

- ***Upward moves*** (destination above source): weight multiplied by 0.8 — vertical progression is rewarded.
- ***Downward moves*** (destination more than 10px below): weight multiplied by 3.0 — downclimbing is heavily penalised as it works against the objective of ascending.
- ***Near-horizontal traverses***: weight multiplied by 2.0 — lateral moves consume energy without vertical gain.

4.2 A* Pathfinding

The route is found using A* search with a heuristic that combines straight-line distance to the finish node and a quadratic penalty for remaining vertical distance. This guides the search preferentially toward features that are both spatially close and vertically progressive.

Start and Finish: The start node is defined as the lowest, leftmost feature (ground-level entry point). The finish node is the highest, rightmost feature (summit).

Gap Detection: After finding the optimal path, segments where the distance between consecutive holds exceeds the configured maximum reach are flagged as gaps. For each gap, the system suggests an interpolated supplementary hold at the midpoint — an "added hold" — to make the route climbable.

4.3 ML Route Classifier

In addition to the heuristic pathfinding, a gradient boosting classifier was trained to predict which transitions between holds a real climber would choose, based on features extracted from real climbing route data. The feature vector for each candidate transition includes:

- $\Delta x, \Delta y$ (signed and absolute), distance, angle
- Vertical and horizontal movement ratios
- Boolean flags: `is_upward`, `is_lateral`
- Source and target hold attributes (area, solidity, grip type)

Training results:

Dataset	Accuracy	Precision	Recall	F1
Training (n=174)	1.000	1.000	1.000	1.000
Validation (n=59)	0.746	0.500	0.267	0.348

Test (n=59)	0.746	0.500	0.333	0.400
Cross-Val (5-fold)	—	—	—	0.416 ± 0.090

The model achieves 74.6% accuracy on held-out data. The gap between training and validation performance indicates some overfitting, expected given the small dataset size. The most predictive features are vertical displacement (Δy , importance: 0.235), absolute vertical displacement (0.175), and total distance (0.141), confirming that vertical progression and reachability are the primary determinants of move selection in real climbing.

5. Stage 3 — Biomechanics Analysis

A key differentiator of RockHard is that its route planning parameters are not hardcoded constants, but are *learned from real climbing footage*. This grounds the system in human physiology rather than arbitrary thresholds.

5.1 Pose Extraction from Video

Climbing video footage (data/climbing_movements/climbing_movement_01.MOV, _02.MOV) is processed using **MediaPipe Pose Landmarker** (heavy model) to extract 33 skeletal landmarks per frame at full video frame rate. From these, 14 key points are retained: bilateral shoulders, elbows, wrists, hips, knees, ankles, fingertips, and heels. Output is stored as frame-by-frame CSV files with pixel coordinates and confidence scores for each landmark.

Annotated video outputs (outputs/climbing_movement_01_annotated.mp4) overlay the extracted skeleton on the original footage for qualitative verification.

5.2 Learned Biomechanics Parameters

The `learn_biomechanics.py` module analyses the extracted pose sequences to compute reach statistics. From the processed footage, the following parameters were extracted and stored in `outputs/climbing_config.json`:

Parameter	Learned Value
Maximum hand reach	136.3 px
Comfortable hand reach	27.6 px
Maximum foot reach	157.9 px
Comfortable foot reach	25.3 px
Maximum hand-to-foot distance	395.9 px
Shoulder width	54.0 px

Landmark confidence weights (used to downweight uncertain pose detections) were also computed per limb endpoint, with right heel achieving the highest reliability (confidence 1.00) and right index finger the lowest (0.74).

These parameters are passed into the route generation stage to replace default reach thresholds, so the route planner operates within physiologically realistic constraints derived from actual observed climbing behaviour.

5.3 Four-Limb Route Planning

The `limb_route_planner.py` module extends the basic route sequence into a detailed four-limb plan, specifying at each move which of the four limbs (left hand, right hand, left foot, right foot) advances to which hold. The planner maintains a simulated body state and evaluates the strain of each possible limb assignment, seeking to balance the body and minimise over-extension. Outputs are stored as per-route limb plan CSV files (`outputs/results/artificial_wall_route_*_limb_plan.csv`) and visualised as annotated diagrams.

6. Stage 4 — Route Grading

Once a route is generated, it is assessed against the standard **V-scale bouldering grade system** (V0 through V8+). The grading module maps quantitative route metrics to difficulty grades using predefined grade profiles.

6.1 Grade Profiles

Each grade profile specifies:

Grade	Description	Max Reach (% of height)	Permitted Grips	Hold Density
V0	Beginner	45%	Sill, Ledge, Added Hold	1.0×
V1	Easy	55%	Sill, Ledge, Protrusion, Added Hold	0.8×
V2	Moderate	70%	Sill, Ledge, Protrusion, Crimp	0.6×
V3	Intermediate	65%	Ledge, Protrusion, Crimp, Ornament	0.45×
V4	Hard	80%	Protrusion, Crimp, Ornament	0.3×
V5	Expert	100%	Crimp, Ornament	0.2×

6.2 Grading Metrics

The grading algorithm evaluates a route against five dimensions:

- **Move difficulty:** Average grip difficulty across all transitions weighted by move distance.
- **Reach demands:** Maximum single-move distance as a fraction of the floor-to-ceiling height.
- **Hold density:** Number of holds per unit of wall area, reflecting how much rest opportunity the route provides.

- **Angle consistency:** Standard deviation of move angles from vertical — routes with more direct vertical paths are generally easier.
- **Hold variety:** Entropy of grip type distribution — routes relying on a wider variety of grip types are harder.

The resulting grade is used both for reporting to the user and for the supplementary hold suggestion feature, which auto-inserts easier holds to bring a route down to a target grade.

7. Stage 5 — Facade Redesign

The final stage produces a visual representation of what the building facade would look like with climbing features integrated. This is the primary deliverable for the design communication use case.

7.1 Generative Render Pipeline

The redesign pipeline uses **Stable Diffusion 2 Inpainting** ([stabilityai/stable-diffusion-2-inpainting](https://github.com/stabilityai/stable-diffusion-2-inpainting)) executed locally on CUDA (NVIDIA GTX 1060 6GB, FP16 precision) to inpaint a climbing wall structure into the facade image.

Mask Generation: Rather than masking individual hold positions as isolated circles (which produces fragmented, unconvincing results), the mask is generated as a continuous strip along the route path. The route hold positions are connected by a thick polyline (approximately 7% of the shorter image dimension in width), with smooth caps at each hold point. Morphological closing fills gaps, and Gaussian feathering softens the boundary so the inpainted region blends naturally into the surrounding facade.

Prompt Engineering: The inpainting prompt was designed to produce architecturally integrated, photorealistic climbing structures — deliberately avoiding the appearance of plastic gym equipment:

*"Photorealistic architectural photography, same building facade, natural stone and concrete rock climbing wall structure integrated into the building, textured stone surface with embedded climbing holds and footholds, climbing wall panel seamlessly attached to the facade, same wall color palette and material, same lighting and shadows, sharp detail, high resolution, professional photography."**

The negative prompt explicitly excludes plastic holds, bright colours, cartoonish outputs, and structural changes to the original building.

SD Parameters: 35 inference steps, guidance scale 8.5, inpainting strength 0.98.

7.2 Gendo.ai API Integration

As the primary photorealistic rendering backend, the system integrates with the **Gendo.ai API** — a professional architectural AI rendering platform used by firms including Zaha Hadid Architects and KPF. Given a facade image and a descriptive prompt, the Gendo.ai API returns a high-fidelity architectural render with the requested climbing features integrated into the building. Renders were generated for the two primary test facades — `facade_02` (a multi-storey residential building) and `facade_modern_apartment_01` (a contemporary apartment facade) — across multiple prompt and style variations. The API call is triggered from the application backend and the resulting image is retrieved, cached locally, and displayed in the viewport through a multi-step progress interface that communicates the generation stages to the user. The prompt sent to the API was carefully engineered to produce

architecturally coherent results: specifying natural stone and concrete materials, same-building colour palette preservation, and the exclusion of plastic gym-style holds.

7.3 OpenCV Compositing Fallback

If GPU or diffusers are unavailable, the system falls back to an OpenCV-based compositing approach. The wall colour is sampled from the local region around the route, and a stone-textured strip is rendered by blending a slightly desaturated, noise-perturbed version of the wall colour along the route path, with additional simulated hold bumps (highlight/shadow pairs) at each hold position. While lower fidelity than SD inpainting, this fallback produces a plausible indication of where the climbing structure would be situated.

8. Web Application

The full pipeline is exposed through **RockHard**, an interactive web application built with Flask (Python backend) and a single-page HTML/JavaScript frontend served at `http://localhost:5002`.

8.1 Interface Overview

The application is structured as a three-panel layout:

- **Left panel:** Controls for image selection/upload, hold placement tools, generate actions, and grade targeting.
- **Centre viewport:** A full-resolution interactive canvas displaying the facade image with hold overlays, route lines, and analysis results. Navigable via a tab bar: *Original, Depth, Protrusion, Route, New Route, Redesign*.
- **Right panel:** Numerical results (hold count, grade, move count, biomechanics data), route step list, hold gap analysis, and suggested supplementary holds.

8.2 Core Workflows

Auto-Analysis Workflow:

6. User uploads or selects a facade photograph.
7. The system automatically runs the full detection pipeline (CV + depth + YOLO + ensemble).
8. Detected holds are overlaid on the canvas as colour-coded circles by grip type.
9. A* pathfinding generates an initial route.
10. The route is graded and displayed with V-scale difficulty.
11. Gap analysis identifies unreachable transitions and suggests supplementary holds.
12. User can accept suggested holds, re-route, and adjust target grade.

Manual Annotation Workflow:

13. User clicks directly on the canvas to place holds of selected grip types and sizes.
14. Undo/clear controls allow iterative placement.
15. A "Generate Route from Holds" action runs A* on the user-placed holds.
16. Same grading and gap analysis as the auto workflow.

Render Climbing Panels:

Along a user-defined path (sequence of placed holds), the system renders coloured climbing panels — interpolating hold positions along the path at a user-configured spacing, assigning hold types, and compositing them onto the facade image with physically-based shading (shadow, highlight, specular).

Realistic Facade Render:

Triggers the AI facade redesign pipeline, producing a photorealistic image of the building with a climbing wall integrated along the route path. A multi-step progress indicator communicates the generation stages to the user. The result is displayed full-size in the Redesign tab of the canvas viewport and is available for download.

Grade Adjustment:

After analysis, users can select a target grade (V0–V5). The system automatically computes the minimum number of supplementary holds required to make the current route achievable at the selected grade, inserts them, re-runs the route, and visualises the updated result.

3D View:

A Three.js-powered 3D modal projects the facade image onto a mesh derived from the depth map, providing a three-dimensional spatial context for the detected route. The route path and hold positions are rendered on the 3D surface.

8.3 API Endpoints

The backend exposes a RESTful JSON API:

Endpoint	Method	Description
`/api/images`	GET	List available facade images
`/api/upload`	POST	Upload a new facade image
`/api/image/<name>`	GET	Serve a facade image
`/api/analyze`	POST	Run full detection + route pipeline
`/api/route_from_holds`	POST	Run A* on user-placed holds
`/api/render_route`	POST	Render climbing panels along a path
`/api/depth_analyze`	POST	Run depth analysis pipeline
`/api/make_easier`	POST	Insert holds to hit target grade
`/api/generate_redesign`	POST	Run SD inpainting redesign
`/api/fake_redesign`	POST	Trigger Gendo.ai API and return rendered facade
`/api/render_facade`	POST	Generative facade design (parametric)
`/api/result/<name>`	GET	Serve a result image
`/api/biomechanics`	GET	Return learned biomechanics config

9. Technical Stack

Component	Technology
Language	Python 3.10+

Web framework	Flask
Frontend	HTML5 Canvas, Vanilla JS, Three.js
Image processing	OpenCV 4.8+
Pose estimation	MediaPipe Pose Landmarker (heavy model)
Depth estimation	Depth Anything V2, MoGe
Object detection	YOLOv8 (Ultralytics)
ML classification	scikit-learn (Gradient Boosting)
Graph & pathfinding	NetworkX (A* search)
Generative AI	Stable Diffusion 2 Inpainting (diffusers, HuggingFace)
Deep learning runtime	PyTorch 2.x, CUDA FP16
3D visualisation	Three.js
Data handling	NumPy, Pandas

10. Results

10.1 Detection Performance

The ensemble detection system was validated against manually annotated ground-truth hold sets on four test facades. The route model achieved the following on held-out data:

- **Test accuracy:** 74.6%
- **Test F1 score:** 0.40
- **Cross-validation F1 (5-fold):** 0.416 ± 0.090

The direction consistency metric (proportion of route transitions in the upward direction) reached 1.0, confirming that the pathfinding correctly enforces vertical progression at all times.

The gap between training accuracy (100%) and validation accuracy (74.6%) reflects overfitting due to the limited size of the annotated dataset. The feature importance analysis reveals that geometric movement features — particularly vertical displacement (Δy , 23.5%) and total distance (14.1%) — are far more predictive than hold appearance features (area, solidity, grip type), which had near-zero importance. This suggests that route selection is primarily driven by spatial geometry rather than hold type in the current training data.

10.2 Biomechanics Learning

From two climbing video sequences, the learned biomechanics configuration captured the following key parameters (in image-space pixels, image-normalised):

- Maximum hand reach: 136.3 px
- Maximum foot reach: 157.9 px
- Maximum hand-to-foot separation: 395.9 px

These learned values replace hardcoded defaults in the route planner, producing routes calibrated to observed human climbing physiology rather than arbitrary constants.

10.3 Route Generation

Across test facades, the system generated navigable routes with complete vertical coverage of the facade. On facades where gaps exceed the maximum reach, supplementary hold suggestions are generated and automatically positioned to bridge the gap. The route visualisation clearly distinguishes existing architectural features from system-suggested added holds through colour coding.

10.4 Facade Redesign

Using both the local SD inpainting pipeline and the Gendo.ai API, the system produces photorealistic visualisations of facades with integrated climbing wall structures. Gendo.ai outputs consistently demonstrated superior photorealism and architectural coherence, making it the preferred rendering path for final outputs. The connected-strip mask approach used for the local SD pipeline produces a cohesive climbing wall panel appearance — a continuous rock or concrete surface running along the route path — rather than isolated spots, which more accurately represents how a real climbing installation would look architecturally.

11. Discussion

11.1 Design Decisions

Ensemble detection over single-method: No single detection method performs reliably across the full range of facade types. CV detection works well on facades with high contrast and defined edges but misses subtly projecting features. Depth-based detection captures geometry but is sensitive to depth model quality. YOLO provides strong prior knowledge from labelled examples but is limited by training set size. The ensemble combines the complementary strengths of each approach.

Graph-based routing over learning-only: A pure learning approach to route generation would require a large corpus of annotated facade climbing routes, which does not exist. The graph/A* formulation encodes structural domain knowledge (vertical progression, reach limits, grip difficulty) directly, providing reliable route quality even with minimal training data. The ML route classifier supplements this with data-driven preferences where training data is available.

Strip-based inpainting mask: Early iterations used individual circular masks around each hold. The resulting inpainted images showed disconnected blobs of different texture at each hold position, failing to produce the appearance of an integrated climbing installation. Switching to a connected strip mask prompted the diffusion model to generate a coherent structural element — a climbing wall panel — which matches how such installations appear in real buildings.

11.2 Limitations

- **Dataset size:** The YOLO and route ML models are trained on small datasets (15–20 images for YOLO; 174 training samples for the route classifier). Larger annotated datasets would substantially improve detection precision and route prediction quality.
- **Depth model dependency:** Depth Anything V2 requires significant GPU memory and time. On lower-specification machines, the Laplacian fallback produces lower-quality depth maps, reducing protrusion detection accuracy.

- **Scale calibration:** The system operates in image-pixel space rather than metric space. Biomechanics reach parameters learned from video are image-pixel values that depend on the distance and angle of the camera during filming, limiting direct physical interpretability.
- **Single-view limitation:** The depth estimation and protrusion detection operate on a single input image. Multi-view sequences (available in `data/3dgs/`) could provide more accurate 3D geometry but are not yet fully integrated into the main pipeline.
- **Facade type specificity:** The detection system and grade profiles were designed and tested on contemporary architectural facades. Performance on highly irregular facades (e.g. heritage buildings with complex ornamental patterns) has not been fully validated.

12. Conclusion

RockHard demonstrates that building facade climbing — an activity that until now has been assessed purely through human expert judgement on-site — can be systematically analysed, planned, and visualised through a machine learning pipeline. The system integrates five detection modalities, graph-based A* pathfinding, biomechanics learned from real climbing video, a quantitative V-scale grading system, and generative AI redesign into a unified, interactive tool accessible through a web browser.

The project contributes a novel formulation of adaptive urban climbing as a structured ML problem, an ensemble feature detection architecture suited to architectural imagery, and a biomechanics-informed pathfinding system that roots route quality assessments in real human physiology. The generative redesign component provides an immediately accessible and visually compelling communication tool for architects, builders, and developers.

Future work would focus on expanding the training datasets, integrating multi-view 3D reconstruction fully into the detection pipeline, introducing physical scale calibration, and extending the grading system to more diverse facade typologies.

Appendix A — Repository Structure

```
RockHard/
├── app.py                # Main Flask web application
├── webapp.py             # Alternative visualiser (legacy)
├── CLAUDE.md             # Project context and conventions
├── requirements.txt      # Python dependencies
├──
├── src/                  # Core library modules
│   ├── hold_detection.py # CV-based feature detection
│   ├── depth_analysis.py # Depth-based detection & full pipeline
│   ├── protrusion_detection_3d.py # 3D geometry protrusion analysis
│   ├── yolo_hold_detection.py # YOLOv8 training & inference
│   ├── color_hold_detection.py # Colour-based segmentation
│   ├── combined_hold_detection.py # Ensemble fusion
│   ├── terrain_segmentation.py # Sky/facade/ground classification
│   ├── route_generation.py # Graph construction & A* pathfinding
│   ├── route_model.py     # ML route transition classifier
│   ├── climbing_grades.py # V-scale grading system
│   ├── pose_extraction.py # MediaPipe video pose extraction
│   └── learn_biomechanics.py # Biomechanics parameter learning
```

```

├── limb_route_planner.py      # Four-limb assignment planner
├── facade_redesign.py        # Stable Diffusion img2img redesign
├── generate_redesign.py      # SD inpainting with strip mask
├── route_render.py           # Route & hold visualisation
├── facade_generator.py       # Procedural facade design
├── data/
│   ├── facade_images/       # Input building photographs
│   ├── facade_features/     # Detected feature CSVs
│   ├── climbing_movements/  # Climbing video & extracted poses
│   ├── references/          # Renders retrieved from Gendo.ai API
│   └── yolo_dataset/        # YOLO training data
├── models/                  # Trained model weights
│   ├── yolo_hold_detector.pt
│   ├── route_model.pkl
│   └── hold_classifier.pkl
├── outputs/                 # All generated artefacts
│   ├── depth_maps/
│   ├── results/
│   ├── redesigns/
│   └── *.json / *.png / *.csv
├── templates/
│   └── main.html            # Single-page web application UI

```

Appendix B — Grip Type Difficulty Reference

Grip Type	Difficulty	Description
Added Hold	0.8	Supplementary bolt-on holds placed to bridge gaps
Sill	1.0	Wide horizontal ledges (windowsills, string courses)
Ledge	2.0	Narrower projecting ledges
Protrusion	2.5	General surface protrusions
Crimp	3.0	Thin horizontal edges requiring finger strength
Pipe	3.5	Cylindrical features (exposed pipes, rails)
Ornament	4.0	Decorative architectural details (bosses, brackets, capitals)